



INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
DELEGAÇÃO REGIONAL DE LISBOA E VALE DO TEJO
CENTRO DE EMPREGO E FORMAÇÃO PROFISSIONAL DE LISBOA

**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

Modulo: UC603

Formador: João Galamba



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

Modulo: UC603.....	1
1. Introdução	2
2. Objectivos	3
3. Desenho e estrutura	3
4. Implementação.....	4
5. Resultados da Execução	20
6. Aprendizagens Adquiridas.....	20
7. Conclusão	21
8. Bibliografia e Recursos	22



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

1. Introdução

Este relatório técnico descreve o processo de conceção, modelação, programação e segurança da infraestrutura relacional que suporta a plataforma de e-commerce **BuyPy**. O desenvolvimento deste projeto enquadra-se no âmbito das unidades curriculares de Modelação de Bases de Dados e Programação em SQL, sob a orientação do formador João Galamba. O foco central do trabalho consistiu em transitar de um modelo conceitual abstrato para uma implementação física rigorosa em MySQL, capaz de gerir catálogos complexos de produtos, contas de utilizadores e fluxos transacionais de faturação.

A motivação primordial para a arquitetura desenhada prende-se com a necessidade de descentralizar a inteligência operacional do negócio, retirando o peso lógico das aplicações cliente e movendo-o diretamente para o motor relacional. Para atingir este patamar de robustez, o ecossistema foi estruturado em torno de três eixos fundamentais: a garantia da integridade referencial em cascata, a validação algorítmica de dados à cabeceira do servidor através de gatilhos operacionais (Triggers) e o isolamento de privilégios através de políticas de controlo de acessos baseados em perfis. Esta abordagem assegura que as regras comerciais e fiscais da organização se mantêm invioláveis, independentemente do canal externo que interaja com o repositório de dados.

2. Objectivos

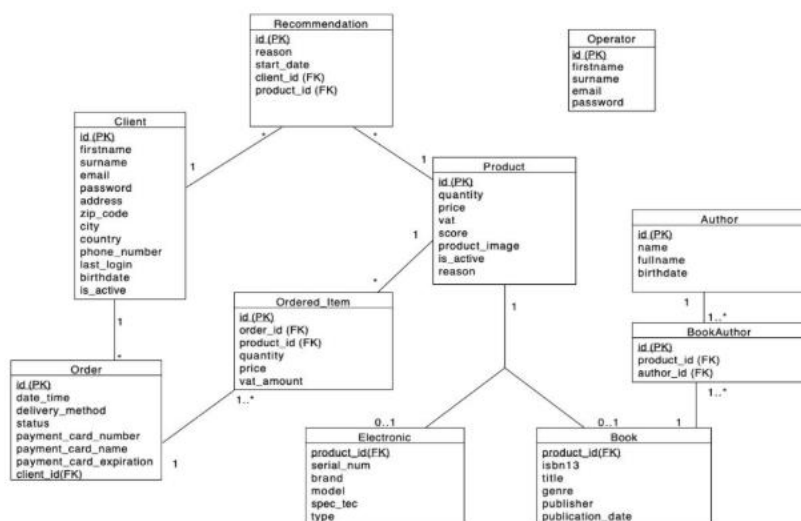
- Desenvolver um modelo relacional normalizado com chaves primárias e estrangeiras consolidadas.
- Implementar mecanismos de automação nativos através de Triggers e Stored Procedures.
- Garantir a higienização de dados diretamente no motor SQL utilizando Expressões Regulares (Regex).
- Configurar uma matriz de segurança granular baseada em perfis de acesso.



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

3. Desenho e estrutura

MODELO DE DADOS





Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01

4. Implementação

- **4.1. Camada de Armazenamento e Restrições de Integridade.**

A criação do esquema físico obedeceu a um processo rigoroso de normalização e tipificação, onde cada tabela foi desenhada com restrições específicas para blindar o repositório contra dados corrompidos. A estrutura detalhada divide-se da seguinte forma:

```
DROP DATABASE IF EXISTS BuyPy;  
CREATE DATABASE BuyPy;  
USE BuyPy;
```

- **Tabela Operator :**

Centraliza os perfis administrativos do BackOffice. Para além de exigir campos obrigatórios (NOT NULL), possui uma restrição CHECK baseada em Expressões Regulares Estendidas (ERE) na coluna email. Esta validação garante, a nível de motor SQL, que nenhum operador é inserido com um endereço eletrónico sintaticamente inválido ou mal formatado.

```
DROP TABLE IF EXISTS Operator;  
CREATE TABLE Operator (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  firstname VARCHAR(250) NOT NULL,  
  surname VARCHAR(250) NOT NULL,  
  email VARCHAR(50) NOT NULL UNIQUE,  
  `password` VARCHAR(250) NOT NULL,  
  CONSTRAINT Operator_Email_CHK CHECK (  
    email REGEXP '^[a-z0-9!#$%&\'*+/?^_`{|}~]+(\.[a-z0-9!#$%&\'*+/?^_`{|}~]+)  
    *@([a-z0-9]([a-z0-9]*[a-z0-9])?\.[a-z0-9]([a-z0-9]*[a-z0-9])?)$'  
  )  
) ;
```



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

- **Tabela Client :**

Armazena as informações de perfil dos compradores. O campo zip_code foi otimizado com o tipo SMALLINT para reduzir o consumo de memória em disco. A coluna country possui um valor por omissão (DEFAULT 'Portugal'), simplificando os novos registos. Adicionalmente, a integridade dos dados é assegurada por duas restrições CHECK: uma que valida a sintaxe do e-mail por Regex e outra (Client_Phone_CHK) que recorre ao padrão `^[0-9]{6,15}$` para forçar que o contacto telefónico contenha estritamente dígitos numéricos.

```
CREATE TABLE `Client` (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  firstname VARCHAR(250) NOT NULL,  
  surname VARCHAR(250) NOT NULL,  
  email VARCHAR(50) NOT NULL UNIQUE,  
  `password` VARCHAR(250) NOT NULL,  
  address VARCHAR(100) NOT NULL,  
  zip_code SMALLINT NOT NULL,  
  city VARCHAR(30) NOT NULL,  
  country VARCHAR(30) NOT NULL DEFAULT 'Portugal',  
  phone_number VARCHAR(15) NOT NULL,  
  last_login TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  birthdate DATE NOT NULL,  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  CONSTRAINT Client_Email_CHK CHECK (  
    email REGEXP '^[a-z0-9!#$%&\'*+/?^_`{|}~-]+  
    (\\.[a-z0-9!#$%&\'*+/?^_`{|}~-]+)  
    *@([a-z0-9]([a-z0-9-]*[a-z0-9])?\\..)  
    +[a-z0-9]([a-z0-9-]*[a-z0-9])?$'  
  ),  
  CONSTRAINT Client_Phone_CHK CHECK (  
    phone_number REGEXP '^[0-9]{6,15}$'  
  )  
);
```



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

- **Tabela Product :**

Atua como a superclasse central da loja. Todas as colunas financeiras e de inventário foram blindadas contra valores corrompidos através de restrições nativas: a quantidade em stock está protegida por CHECK (quantity >= 0), o preço de venda por CHECK (price >= 0) e a taxa fiscal por CHECK (vat BETWEEN 0 AND 100). O campo score restringe a classificação do produto a uma escala de satisfação de 1 a 5, enquanto o campo reason (do tipo TEXT) fica reservado para auditorias de estado.

```
DROP TABLE IF EXISTS Product;
CREATE TABLE Product (
  id          VARCHAR(10) PRIMARY KEY,
  quantity    INT NOT NULL CHECK (quantity >= 0),
  price       DECIMAL(10,2) NOT NULL CHECK (price >= 0),
  vat         DECIMAL(5,2) NOT NULL CHECK (vat BETWEEN 0 AND 100),
  score       INT CHECK (score BETWEEN 1 AND 5),
  product_image VARCHAR(255) NOT NULL,
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  reason      TEXT
);
```

- **Tabela Book :**

Representa a herança exclusiva para o catálogo de livraria. O campo product_id funciona em simultâneo como Chave Primária e Chave Estrangeira ligada à tabela Product, configurado com ON DELETE CASCADE e ON UPDATE CASCADE. Esta regra referencial garante que, se um produto base for removido, o respetivo livro é apagado em simultâneo sem deixar registos órfãos. A coluna isbn13 possui uma restrição CHECK via Regex para assegurar a formatação regulamentar de identificação bibliográfica.



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

```
DROP TABLE IF EXISTS Book;
CREATE TABLE Book (
  product_id VARCHAR(10) PRIMARY KEY,
  isbn13 VARCHAR(20) NOT NULL UNIQUE,
  title VARCHAR(50) NOT NULL,
  genre VARCHAR(50) NOT NULL,
  publisher VARCHAR(100) NOT NULL,
  publication_date DATE NOT NULL,
  CONSTRAINT Book_Product_FK FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT Book_ISBN_CHK CHECK (
    isbn13 REGEXP '^[A-Z0-9-]+$' AND
    NOT (isbn13 LIKE '%-' OR isbn13 LIKE '%-')
  )
);
```

- **Tabela Electronic :**

Representa a herança exclusiva para dispositivos tecnológicos. Aplica a mesma lógica de especialização referencial (ON DELETE CASCADE) na sua Chave Primária product_id. A coluna serial_num foi tipificada como BIGINT para acomodar com segurança números de série extensos e possui uma restrição UNIQUE para impedir a duplicação de equipamentos idênticos na base de dados.

```
DROP TABLE IF EXISTS Electronic;
CREATE TABLE Electronic (
  product_id VARCHAR(10) PRIMARY KEY,
  serial_num BIGINT NOT NULL UNIQUE,
  brand VARCHAR(20) NOT NULL,
  model VARCHAR(20) NOT NULL,
  spec_tec TEXT,
  `type` VARCHAR(10) NOT NULL,
  CONSTRAINT Electronic_Product_FK FOREIGN KEY (product_id) REFERENCES Product(id) ON DELETE CASCADE ON UPDATE CASCADE
);
```

- **Tabela Author :**

Tabela de suporte que armazena os dados biográficos dos criadores das obras literárias. Utiliza uma diferenciação estrita de colunas para otimizar pesquisas: o campo name guarda o nome literário/artístico (geralmente mais curto) e o campo fullname armazena o nome civil completo do autor, sendo a data de nascimento (birthdate) obrigatória para controlo de homónimos.



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

```
DROP TABLE IF EXISTS Author;  
CREATE TABLE Author (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  `name` VARCHAR(100) NOT NULL,  
  fullname VARCHAR(100) NOT NULL,  
  birthdate DATE NOT NULL  
);
```

- **Tabela BookAuthor:**

Resolve a relação de muitos-para-muitos entre Livros e Autores (visto que um livro pode ter vários autores e um autor pode escrever vários livros). A sua Chave Primária é **composta** pelas colunas (product_id, author_id). Ambas as colunas atuam como chaves estrangeiras com integridade em cascata total, garantindo que qualquer alteração ou eliminação nas tabelas de origem se reflete instantaneamente nesta tabela de ligação.

```
DROP TABLE IF EXISTS BookAuthor;  
CREATE TABLE BookAuthor (  
  product_id VARCHAR(10) NOT NULL,  
  author_id INT NOT NULL,  
  PRIMARY KEY (product_id, author_id),  
  CONSTRAINT BookAuthor_Book_FK FOREIGN KEY (product_id) REFERENCES Book(product_id) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT BookAuthor_Author_FK FOREIGN KEY (author_id) REFERENCES Author(id) ON DELETE CASCADE ON UPDATE CASCADE  
);
```

- **Tabela Order :**

Centraliza o registo das transações. A coluna date_time gera automaticamente a marca temporal da compra através de DEFAULT CURRENT_TIMESTAMP. A segurança lógica é garantida por restrições de domínio estritas: o método de entrega é validado por CHECK (delivery_method IN ('regular', 'urgent')) e o estado do fluxo comercial por CHECK (status IN ('open', 'processing', 'pending', 'closed', 'cancelled')). A tabela faz a ligação ao cliente através da chave estrangeira client_id.



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

```
DROP TABLE IF EXISTS `Order`;  
CREATE TABLE `Order` (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  date_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  delivery_method VARCHAR(10) NOT NULL DEFAULT 'regular',  
  `status` VARCHAR(10) NOT NULL DEFAULT 'open',  
  payment_card_number BIGINT NOT NULL,  
  payment_card_name VARCHAR(20) NOT NULL,  
  payment_card_expiration DATE NOT NULL,  
  client_id INT NOT NULL,  
  CONSTRAINT Order_Client_FK FOREIGN KEY (client_id) REFERENCES Client(id) ON UPDATE CASCADE,  
  CONSTRAINT Order_Delivery_CHK CHECK (delivery_method IN ('regular', 'urgent')),  
  CONSTRAINT Order_Status_CHK CHECK (`status` IN ('open', 'processing', 'pending', 'closed', 'cancelled'))  
);
```

- **Tabela Ordered_Item :**

Entidade associativa que mapeia os produtos incluídos em cada encomenda. A sua Chave Primária é **composta** por (order_id, product_id). Para congelar o histórico comercial contra alterações futuras no catálogo, esta tabela duplica de forma segura as colunas price e vat_amount no momento da compra. Contém ainda uma restrição CHECK (quantity > 0) para impedir a inserção de linhas com quantidades nulas ou negativas.

```
DROP TABLE IF EXISTS Ordered_Item;  
CREATE TABLE Ordered_Item (  
  order_id INT NOT NULL,  
  product_id VARCHAR(10) NOT NULL,  
  quantity INT NOT NULL CHECK (quantity > 0),  
  price DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
  vat_amount DECIMAL(10,2) NOT NULL CHECK (vat_amount >= 0),  
  PRIMARY KEY (order_id, product_id),  
  CONSTRAINT Item_Order_FK FOREIGN KEY (order_id) REFERENCES `Order`(id) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT Item_Product_FK FOREIGN KEY (product_id) REFERENCES Product(id) ON UPDATE CASCADE  
);
```

- **Tabela Recommendation (Marketing Relacional)**

Liga o perfil de um cliente a um produto sugerido pelo sistema de marketing. Contém colunas obrigatórias para armazenar a data de início da campanha (start_date) e a justificação comercial da proposta (reason). As suas chaves estrangeiras possuem a regra ON UPDATE CASCADE para garantir a consistência caso os identificadores dos clientes ou produtos sofram modificações.



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

```
DROP TABLE IF EXISTS Recommendation;  
> CREATE TABLE Recommendation (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  reason      VARCHAR(500) NOT NULL,  
  start_date  DATE NOT NULL,  
  client_id   INT NOT NULL,  
  product_id  VARCHAR(10) NOT NULL,  
  CONSTRAINT Rec_Client_FK FOREIGN KEY (client_id) REFERENCES Client(id) ON UPDATE CASCADE,  
  CONSTRAINT Rec_Product_FK FOREIGN KEY (product_id) REFERENCES Product(id) ON UPDATE CASCADE  
- );
```



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

4.2. Camada Lógica de Integridade (Triggers):

Os gatilhos foram programados no motor MySQL com o delimitador BEFORE para intercepar e analisar os dados antes de qualquer escrita física no disco, atuando como a primeira linha de defesa contra dados corrompidos:

- **Gatilho de Complexidade e Cifragem de Passwords**

(trg_Client_Password_Insert):

- **O que faz:** Monitoriza a tabela Client antes de cada inserção.
- **Lógica detalhada:** Utiliza a instrução REGEXP para obrigar a palavra-passe a cumprir as diretivas estritas de segurança (mínimo de 6 caracteres, pelo menos uma letra maiúscula, uma minúscula, um dígito e um símbolo especial da lista !, \$, #, ?, %). Caso a string não cumpra todos os critérios, o trigger dispara um SIGNAL SQLSTATE '45000' com uma mensagem personalizada e **aborta a transação**. Se a senha for segura, o gatilho aplica nativamente a função SHA2(NEW.password, 256). Isto assegura que as passwords são guardadas cifradas de forma irreversível e nunca em texto limpo.

```
DELIMITER //  
  
CREATE TRIGGER trg_Client_Password_Insert  
BEFORE INSERT ON Client  
FOR EACH ROW  
BEGIN  
    IF LENGTH(NEW.password) < 6  
    OR NEW.password NOT REGEXP '[a-z]'  
    OR NEW.password NOT REGEXP '[A-Z]'  
    OR NEW.password NOT REGEXP '[0-9]'  
    OR NEW.password NOT REGEXP '[!$#?%]' THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Erro: Password deve ter 6+ carateres, 1 maiúscula, 1 minúscula, 1 dígito e 1 símbolo (!,$,#,?,%).';  
    END IF;  
    SET NEW.password = SHA2(NEW.password, 256);  
END //
```

- **Gatilho de Controlo de Estados do Catálogo**

(trg_Product_Inactivation_Check):

- **O que faz:** Monitoriza as atualizações (BEFORE UPDATE) na tabela Product.
- **Lógica detalhada:** Avalia a transição de estado da flag is_active. Se um operador tentar desativar um produto (mudando de TRUE para



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

FALSE), o trigger verifica se o campo `reason` foi preenchido. Se o motivo estiver vazio ou nulo, o motor SQL bloqueia a operação para garantir a rastreabilidade do inventário. Se o produto for reativado (`is_active` passa a TRUE), o trigger limpa automaticamente o campo `reason` definindo-o como NULL, mantendo o registo limpo.

```
CREATE TRIGGER trg_Product_Inactivation_Check
BEFORE UPDATE ON Product
FOR EACH ROW
BEGIN
    IF OLD.is_active = TRUE AND NEW.is_active = FALSE THEN
        IF NEW.reason IS NULL OR NEW.reason = '' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Erro: É obrigatório definir uma razão descritiva para inativar o produto.';
        END IF;
    ELSEIF NEW.is_active = TRUE THEN
        SET NEW.reason = NULL;
    END IF;
END //
```



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

4.3. Camada de Abstração e Operações de Negócio (Stored Procedures):

As rotinas armazenadas encapsulam os fluxos comerciais complexos dentro do próprio servidor, garantindo máxima performance e impedindo que aplicações externas executem queries erradas:

- **Procedimento ProductByType:**
 - **O que faz:** Filtra e apresenta o catálogo da loja com base no tipo de artigo selecionado (Livro ou Electronica) [INDEX: 0.1.2].
 - **Lógica detalhada:** Executa uma junção externa (LEFT JOIN) entre a tabela pai Product, as tabelas filhas Book e Electronic, e a tabela Recommendation. Através de uma estrutura CASE WHEN, a rotina identifica de forma dinâmica a que especialização pertence o ID do produto e devolve uma string legível ("Livro", "Electronica"). Usa também a função IF para converter a flag binária de recomendações num texto simplificado ("Sim/Não"), facilitando a leitura imediata dos dados.



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

```
CREATE PROCEDURE ProductByType(IN param_tipo VARCHAR(20))
> BEGIN
    SELECT
        p.id AS codigo_produto,
        p.price AS preco,
        p.score AS pontuacao,
        IF(r.product_id IS NOT NULL, 'Sim', 'Não') AS recomendado,
        IF(p.is_active, 'Activo', 'Inactivo') AS estado,
        p.product_image AS ficheiro,
    > CASE
        WHEN b.product_id IS NOT NULL THEN 'Livro'
        WHEN e.product_id IS NOT NULL THEN 'Electronica'
        ELSE 'Desconhecido'
    ~ END AS tipo_produto
    FROM Product p
    LEFT JOIN Book b ON p.id = b.product_id
    LEFT JOIN Electronic e ON p.id = e.product_id
    LEFT JOIN (SELECT DISTINCT product_id FROM Recommendation) r ON p.id = r.product_id
    WHERE param_tipo IS NULL
        OR (param_tipo = 'Livro' AND b.product_id IS NOT NULL)
        OR (param_tipo = 'Electronica' AND e.product_id IS NOT NULL);
~ END //
```

- **Procedimento CreateOrder**

- **O que faz:** Inicializa um novo processo de compra na tabela Order.
- **Lógica detalhada:** Recebe os dados do cliente, o método de entrega e as informações do cartão de pagamento. A rotina executa o INSERT definindo o estado inicial por omissão como 'open'. Esta centralização garante que todas as encomendas nascem sob as mesmas regras estruturais, sem depender de scripts externos.

```
• CREATE PROCEDURE CreateOrder(
    IN param_client_id INT, IN param_metodo VARCHAR(10),
    IN param_num_cartao BIGINT, IN param_nome_cartao VARCHAR(20), IN param_validade DATE
)
• BEGIN
    INSERT INTO `Order` (client_id, delivery_method, payment_card_number, payment_card_name, payment_card_expiration, `status`)
    VALUES (param_client_id, param_metodo, param_num_cartao, param_nome_cartao, param_validade, 'open');
    END //
```



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

- **Procedimento AddProductToOrder**

- **O que faz:** Adiciona itens à encomenda e processa a vertente fiscal de forma relacional.
- **Lógica detalhada:** É a rotina mais crítica do negócio. Ao receber o ID da encomenda, o código do produto e a quantidade desejada, ela executa uma subconsulta interna para capturar o preço atual e a percentagem de IVA (vat) registada na tabela Product. De seguida, calcula o valor absoluto do imposto com a fórmula: $(\text{Preço} * (\text{Percentagem_IVA} / 100)) * \text{Quantidade}$. Por fim, insere a linha na tabela Ordered_Item. Isto blinda o sistema contra erros de faturação humanos ou de software.

```
CREATE PROCEDURE AddProductToOrder(IN param_order_id INT, IN param_product_id VARCHAR(10), IN param_qtd INT)
BEGIN
    DECLARE v_price DECIMAL(10,2);
    DECLARE v_vat_pct DECIMAL(5,2);
    DECLARE v_vat_amount DECIMAL(10,2);

    SELECT price, vat INTO v_price, v_vat_pct FROM Product WHERE id = param_product_id;
    SET v_vat_amount = (v_price * (v_vat_pct / 100)) * param_qtd;

    INSERT INTO Ordered_Item (order_id, product_id, quantity, price, vat_amount)
    VALUES (param_order_id, param_product_id, param_qtd, v_price, v_vat_amount);
END //
```

- **Procedimento GetOrderTotal**

- **O que faz:** Realiza o fecho e o somatório contabilístico de uma encomenda.
- **Lógica detalhada:** Percorre as linhas da tabela Ordered_Item associadas à encomenda e recorre à função agregadora SUM para somar o valor de multiplicação $(\text{price} * \text{quantity}) + \text{vat_amount}$. A função COALESCE é aplicada para garantir que, caso a encomenda esteja vazia, a rotina devolva 0.00 em vez de um valor nulo (NULL). O resultado final é injetado diretamente num parâmetro de saída OUT DECIMAL(10,2).



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

```
CREATE PROCEDURE GetOrderTotal(IN param_order_id INT, OUT total_montante DECIMAL(10,2))
BEGIN
    SELECT COALESCE(SUM((price * quantity) + vat_amount), 0.00) INTO total_montante
    FROM Ordered_Item
    WHERE order_id = param_order_id;
END //
```

- **Procedimento DailyOrders**

- **O que faz:** Permite a auditoria diária das vendas feitas na plataforma.
- **Lógica detalhada:** Recebe um parâmetro do tipo DATE. Como a coluna date_time armazena carimbos completos (com horas, minutos e segundos), a procedure aplica a função DATE(date_time) para isolar apenas o dia, permitindo listar com precisão absoluta todas as compras efetuadas naquela data específica.

```
CREATE PROCEDURE DailyOrders(IN param_data DATE)
BEGIN
    SELECT id, date_time, delivery_method, `status`, client_id
    FROM `Order`
    WHERE DATE(date_time) = param_data;
END //
```

- **Procedimento AnnualOrders**

- **O que faz:** Extrai o histórico anual de consumo de um determinado.
- **Lógica detalhada:** Recebe o ID do cliente e o ano civil em formato numérico. A rotina utiliza a função nativa YEAR(date_time) para extrair o ano do carimbo temporal da encomenda e cruza-o com o filtro do cliente, gerando um relatório limpo de faturação anual.

```
CREATE PROCEDURE AnnualOrders(IN param_client_id INT, IN param_ano INT)
BEGIN
    SELECT id, date_time, delivery_method, `status`
    FROM `Order`
    WHERE client_id = param_client_id
    AND YEAR(date_time) = param_ano;
END //
```



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

4.4. Perfis de Segurança e Matriz de Privilégios (users.sql)

A gestão de acessos e a segurança do servidor MySQL foram desenhadas com base no princípio do privilégio mínimo, implementando uma arquitetura **RBAC (Role-Based Access Control)**. Em vez de atribuir permissões diretamente às contas dos utilizadores físicos, foram criados perfis abstratos (Roles) que limitam o raio de ação de cada função na organização:

- **Perfil WEB_CLIENT_ROLE (Aplicação Web/Operacional)**
 - **O que faz:** Controla o nível de acesso que as aplicações externas têm sobre a base de dados.
 - **Lógica detalhada e restrições granulares:**
 - **Leitura Geral:** Concede permissão de consulta (GRANT SELECT) em todas as tabelas para permitir a visualização do catálogo e histórico.
 - **Escrita Transacional:** Permite a inserção e atualização de registos (INSERT, UPDATE) apenas nas tabelas dinâmicas do fluxo comercial (Client, Order, Ordered_Item, Recommendation).
 - **Eliminação Controlada:** Permite remover registos (DELETE) estritamente nas tabelas Order e Ordered_Item, permitindo o cancelamento ou esvaziamento de carrinhos de compras.
 - **Restrição Cirúrgica ao Nível da Coluna:** Para mitigar fraudes comerciais, o perfil **está proibido** de alterar preços ou taxas de imposto. Foi aplicada a instrução GRANT UPDATE (quantity) ON BuyPy.Product, o que significa que a aplicação web apenas tem autorização para decrementar ou incrementar o stock, bloqueando qualquer outra modificação na tabela Product.
 - **Direitos de Execução:** Concede privilégios de execução (GRANT EXECUTE) em todas as Stored Procedures operacionais necessárias para o negócio.



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

```
USE BuyPy;
CREATE ROLE IF NOT EXISTS 'WEB_CLIENT_ROLE';
GRANT SELECT ON BuyPy.* TO 'WEB_CLIENT_ROLE';
GRANT INSERT, UPDATE ON BuyPy.`Client` TO 'WEB_CLIENT_ROLE';
GRANT INSERT, UPDATE ON BuyPy.`Order` TO 'WEB_CLIENT_ROLE';
GRANT INSERT, UPDATE ON BuyPy.Ordered_Item TO 'WEB_CLIENT_ROLE';
GRANT INSERT, UPDATE ON BuyPy.Recommendation TO 'WEB_CLIENT_ROLE';
GRANT DELETE ON BuyPy.`Order` TO 'WEB_CLIENT_ROLE';
GRANT DELETE ON BuyPy.Ordered_Item TO 'WEB_CLIENT_ROLE';
GRANT UPDATE (quantity) ON BuyPy.Product TO 'WEB_CLIENT_ROLE';
GRANT EXECUTE ON PROCEDURE BuyPy.CreateOrder TO 'WEB_CLIENT_ROLE';
GRANT EXECUTE ON PROCEDURE BuyPy.GetOrderTotal TO 'WEB_CLIENT_ROLE';
GRANT EXECUTE ON PROCEDURE BuyPy.AddProductToOrder TO 'WEB_CLIENT_ROLE';
```

- **Perfil BUYDB_OPERATOR_ROLE (Operadores de BackOffice)**
 - **O que faz:** Destina-se ao pessoal administrativo que faz o suporte ao cliente e inventário.
 - **Lógica detalhada:** Concede permissões totais de manipulação de dados (SELECT, INSERT, UPDATE, DELETE) e execução de rotinas em todo o esquema. Contudo, este perfil **não possui direitos de alteração estrutural (DDL)**, impedindo operadores de apagar tabelas ou modificar colunas.

```
CREATE ROLE IF NOT EXISTS 'BUYDB_OPERATOR_ROLE';
GRANT SELECT, INSERT, UPDATE, DELETE, EXECUTE ON BuyPy.* TO 'BUYDB_OPERATOR_ROLE';
```

- **Perfil BUYDB_ADMIN_ROLE (Administrador de Base de Dados)**
 - **O que faz:** Destina-se aos administradores de sistemas (DBA).
 - **Lógica detalhada:** Concede controlo total absoluto sobre a base de dados (GRANT ALL PRIVILEGES), incluindo a capacidade de alterar a estrutura física das tabelas (DDL). Adicionalmente, inclui a cláusula WITH GRANT OPTION, permitindo a este perfil criar novos utilizadores e revogar ou conceder permissões no servidor.

```
GRANT ALL PRIVILEGES ON BuyPy.* TO 'BUYDB_ADMIN_ROLE' WITH GRANT OPTION;
```



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

- **Mapeamento de Utilizadores Físicos**

Para validar esta matriz de segurança em ambiente real, foram criadas três contas de utilizador locais associadas à palavra-passe regulamentar do projeto (Lmxy20#a):

- O utilizador **Wilson** foi associado ao perfil `WEB_CLIENT_ROLE`.
- O utilizador **Alberto** foi associado ao perfil `BUYDB_OPERATOR_ROLE`.
- O utilizador **Orlando** foi associado ao perfil `BUYDB_ADMIN_ROLE`.

Todas as contas utilizam a instrução `SET DEFAULT ROLE` para garantir que, no momento em que fazem login no servidor, o respetivo perfil de segurança é ativado automaticamente, blindando o sistema desde o primeiro segundo de sessão.

```
CREATE USER IF NOT EXISTS 'Wilson'@'%' IDENTIFIED BY 'Lmxy20#a';  
CREATE USER IF NOT EXISTS 'Alberto'@'%' IDENTIFIED BY 'Lmxy20#a';  
CREATE USER IF NOT EXISTS 'Orlando'@'%' IDENTIFIED BY 'Lmxy20#a';
```

```
GRANT 'WEB_CLIENT_ROLE' TO 'Wilson'@'%;  
GRANT 'BUYDB_OPERATOR_ROLE' TO 'Alberto'@'%;  
GRANT 'BUYDB_ADMIN_ROLE' TO 'Orlando'@'%;
```

```
SET DEFAULT ROLE 'WEB_CLIENT_ROLE' TO 'Wilson'@'%;  
SET DEFAULT ROLE 'BUYDB_OPERATOR_ROLE' TO 'Alberto'@'%;  
SET DEFAULT ROLE 'BUYDB_ADMIN_ROLE' TO 'Orlando'@'%;
```

```
FLUSH PRIVILEGES;
```

4.5. Camada de Testes

O ficheiro de dados foi estruturado de forma a simular um ambiente de produção real, servindo como uma suite de testes integrados. O preenchimento das tabelas segue uma ordem sequencial estrita para respeitar as restrições das chaves estrangeiras, dividindo-se nas seguintes fases lógicas: [1]

- **Fase 1: Popularização Base de Utilizadores e Autores**

O script inicia com a inserção dos perfis em `Operator` e dos escritores civis em `Author`. De seguida, são registados os dados em `Client`. Nesta fase,



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

todas as palavras-passe inseridas (ex: 'Antonio123#') contêm letras maiúsculas, minúsculas, números e símbolos. Esta formatação é obrigatória para que os registos passem com sucesso pelo trigger `trg_Client_Password_Insert`, que os valida e converte automaticamente em hashes SHA-256.

USE BuyPy;

```
INSERT INTO Operator (firstname, surname, email, 'password') VALUES
```

```
('Mariana', 'Silva', 'mariana.silva@buypy.pt', 'Mariana123!'),  
( 'Jorge', 'Santos', 'jorge.santos@buypy.pt', 'JorgeSecure#');
```

```
INSERT INTO `Client` (firstname, surname, email, 'password', address, zip_code, city, phone_number, birthdate, is_active) VALUES
```

```
('António', 'Américo', 'antonio@mail.com', 'Antonio123#', 'Rua das Flores, 45', 8200, 'Albufeira', '912345678', '1995-04-12', TRUE),  
( 'Beatriz', 'Bastos', 'beatriz@mail.com', 'Beatriz456$', 'Av. Central, 102', 4700, 'Braga', '923456789', '1998-09-21', TRUE),  
( 'Catarina', 'Costa', 'catarina.costa@mail.com', 'Catarina789?', 'Largo do Rossio, 8', 3000, 'Coimbra', '934567890', '1992-11-05', FALSE),  
( 'Pedro', 'Pacheco', 'pedro.pacheco@mail.com', 'Pedro111%', 'Rua Direita, 14', 4700, 'Braga', '965432109', '2001-02-28', TRUE);
```

```
INSERT INTO Author (name, fullname, birthdate) VALUES
```

```
('Eça de Queirós', 'José Maria de Eça de Queirós', '1845-11-25'),  
( 'José Saramago', 'José de Sousa Saramago', '1922-11-16');
```

- **Fase 2: Instanciação do Catálogo de Artigos**

Para testar a integridade referencial da especialização de produtos, os dados são inseridos em duas etapas obrigatórias. Primeiro, criam-se as identidades genéricas com os preços e as taxas de IVA na tabela `Product` (PROD001 a PROD004). Imediatamente a seguir, os atributos específicos são injetados nas tabelas filhas `Book` (com os respetivos títulos e editoras) e `Electronic` (com as marcas e números de série BIGINT), estabelecendo uma ligação perfeitamente normalizada.



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

```
INSERT INTO Product (id, quantity, price, vat, score, product_image, is_active) VALUES  
( 'PROD001', 120, 18.50, 6.00, 5, 'imagens/maias.jpg', TRUE),  
( 'PROD002', 45, 22.00, 6.00, 4, 'imagens/convento.jpg', TRUE),  
( 'PROD003', 25, 79.99, 23.00, 4, 'imagens/mp3.jpg', TRUE),  
( 'PROD004', 12, 599.00, 23.00, 5, 'imagens/tv4k.jpg', TRUE);
```

```
INSERT INTO Book (product_id, isbn13, title, genre, publisher, publication_date) VALUES  
( 'PROD001', '978-972-004-900-0', 'Os Maias', 'Romance', 'Porto Editora', '1888-01-01'),  
( 'PROD002', '978-989-660-001-2', 'Memorial do Convento', 'Histórico', 'Caminho', '1982-10-01');
```

```
INSERT INTO Electronic (product_id, serial_num, brand, model, spec_tec, `type`) VALUES  
( 'PROD003', 887261543, 'Sony', 'Walkman E', '8GB, Bluetooth, Ecrã 1.77', 'Leitor'),  
( 'PROD004', 991827364, 'Samsung', 'Crystal UHD', '55 polegadas, Smart TV, HDR', 'Televisão');
```

- **Fase 3: Consolidação de Relações Muitos-para-Muitos (M:N)**

O script valida o comportamento das tabelas associativas do modelo. A tabela BookAuthor é povoada cruzando os identificadores dos livros com os IDs dos respetivos autores (ex: associando a obra Os Maias ao registo de Eça de Queirós). Da mesma forma, criam-se registos em Recommendation, ligando de forma calendarizada as preferências dos clientes ativos a produtos em destaque com justificações de marketing obrigatórias (NOT NULL).

```
31 • INSERT INTO BookAuthor (product_id, author_id) VALUES  
32   ( 'PROD001', 1),  
33   ( 'PROD002', 2);  
34  
35 • INSERT INTO Recommendation (reason, start_date, client_id, product_id) VALUES  
36   ( 'Campanha de Clássicos da Literatura Portuguesa', '2026-07-26', 1, 'PROD001'),  
37   ( 'Sugestão com base em preferências de Tecnologia UHD', '2026-07-26', 2, 'PROD004');
```

- **Fase 4: Simulação Transaccional e Automação de Faturação**

A última fase do script serve para auditar o funcionamento prático de toda a camada de Stored Procedures do servidor. Em vez de usar comandos INSERT manuais e expostos a erros nas tabelas de faturação, o povoamento invoca o procedimento CreateOrder para abrir encomendas com dados de cartões de pagamento válidos. Posteriormente, chama a procedure AddProductToOrder, forçando o motor MySQL a consultar o catálogo, capturar o IVA de 6% ou



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

23% e calcular autonomamente os montantes fiscais antes de gravar as linhas em Ordered_Item.

```
CALL CreateOrder(1, 'regular', 4532112233445566, 'António Américo', '2031-01-01');  
CALL AddProductToOrder(1, 'PROD001', 2);  
CALL AddProductToOrder(1, 'PROD003', 1);  
  
CALL CreateOrder(2, 'urgent', 4912665544332211, 'Beatriz Bastos', '2029-06-01');  
CALL AddProductToOrder(2, 'PROD004', 1);
```



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

5. Resultados da Execução:

```
40 • select * from `Client`;
```

Result Grid											
Filter Rows:											
	id	firstname	surname	email	password	address	zip_code	city	country	phone_number	last_login
▶	1	António	Américo	antonio@mail.com	518d80c7ce932f8535f6ce31b2f1bceebe85ba03...	Rua das Flores, 45	8200	Albufeira	Portugal	912345678	2026-08-03 19:
	2	Beatriz	Bastos	beatriz@mail.com	9a269b9b8fb0209fa11c3b933ebd4a9fefed096...	Av. Central, 102	4700	Braga	Portugal	923456789	2026-08-03 19:
	3	Catarina	Costa	catarina.costa@mail.com	55a280f7dfd704e901a005ba9b560e732ba2e81...	Largo do Rossio, 8	3000	Coimbra	Portugal	934567890	2026-08-03 19:
	4	Pedro	Pacheco	pedro.pacheco@mail.com	b47e8782e592618d197824585ea9271797e35b...	Rua Direita, 14	4700	Braga	Portugal	965432109	2026-08-03 19:
*		NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
42 • select * from `Order`;
```

Result Grid							
Filter Rows:							
	id	date_time	delivery_method	status	payment_card_number	payment_card_name	payment_card_expiration
▶	1	2026-08-03 19:33:28	regular	open	4532112233445566	António Américo	2031-01-01
	2	2026-08-03 19:33:28	urgent	open	4912665544332211	Beatriz Bastos	2029-06-01
*		NULL	NULL	NULL	NULL	NULL	NULL

```
44 • select * from Ordered_Item;
```

Result Grid				
Filter Rows:				
	order_id	product_id	quantity	price
▶	1	PROD001	2	18.50
	1	PROD003	1	79.99
	2	PROD004	1	599.00
*	NULL	NULL	NULL	NULL



Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01

6. Aprendizagens Adquiridas

1. **Encapsulamento Relacional:** Compreensão do impacto positivo em delegar cálculos fiscais e validações para o motor SQL, poupando tráfego e garantindo uniformidade.
2. **Segurança por Privilégios Mínimos:** Aplicação prática de restrições de escrita ao nível da coluna, blindando dados sensíveis diretamente no servidor.
3. **Higienização Nativa por Regex:** Utilização avançada de expressões regulares no motor SQL para garantir o cumprimento estrito dos formatos de e-mail e contacto.
4. **Modelação de Especializações:** Implementação prática de herança exclusiva através do mapeamento partilhado de Chaves Primárias.



Curso de Especialização Tecnológica- Técnico de Gestão de redes e Sistemas Computacionais – CET-TEGRSC01

7. Conclusão

A concretização do projeto **BuyPy** constituiu um marco fundamental na consolidação de competências avançadas de arquitetura, administração e programação de sistemas de bases de dados relacionais. A metodologia adotada permitiu transitar com sucesso de uma abordagem puramente declarativa (DDL clássico) para um ambiente dinâmico e auto-sustentável, onde a segurança e as regras de negócio encontram-se blindadas de forma nativa no motor de base de dados.

O balanço do trabalho final destaca-se pelo cumprimento integral e rigoroso dos seguintes pilares estruturais do enunciado:

- **Automação e Abstração Fiscal:** A transferência da lógica de cálculo do IVA para os procedimentos internos (`AddProductToOrder` e `GetOrderTotal`) eliminou a redundância computacional em aplicações cliente, garantindo a atomicidade das transações e reduzindo drasticamente o tráfego na rede local.
- **Segurança Ativa à Cabeceira:** A implementação dos triggers criptográficos e de validação por expressões regulares estendidas (ERE) demonstrou que o servidor SQL é capaz de atuar como a primeira e mais robusta linha de defesa do ecossistema, impedindo a injeção de credenciais fracas ou estados de integridade corrompidos.
- **Governança de Dados com Privilégios Mínimos:** A configuração de papéis orientados a perfis (*RBAC*), acoplada à restrição cirúrgica de alteração de dados ao nível de colunas específicas (`Product.quantity`), traduziu-se num modelo prático de governança que mitiga riscos de fraudes comerciais e acessos indevidos.

Em suma, o esquema relacional desenvolvido estabelece uma infraestrutura escalável, segura e em total conformidade com as boas práticas de engenharia de dados. O projeto demonstra que uma base de dados bem planeada e programada não serve apenas para armazenar registos, mas sim para governar, proteger e processar a informação com total independência de camadas externas de software.



**Curso de Especialização Tecnológica- Técnico de Gestão de redes e
Sistemas Computacionais – CET-TEGRSC01**

8. Bibliografia e Recursos

- Materiais didáticos e fichas de laboratório fornecidos pelo formador João Galamba.
- Plataforma Draw.io para a modelação do diagrama de classes e relações.
- Repositório de código e histórico de versionamento alojados no GitHub.